1 Implementation Details

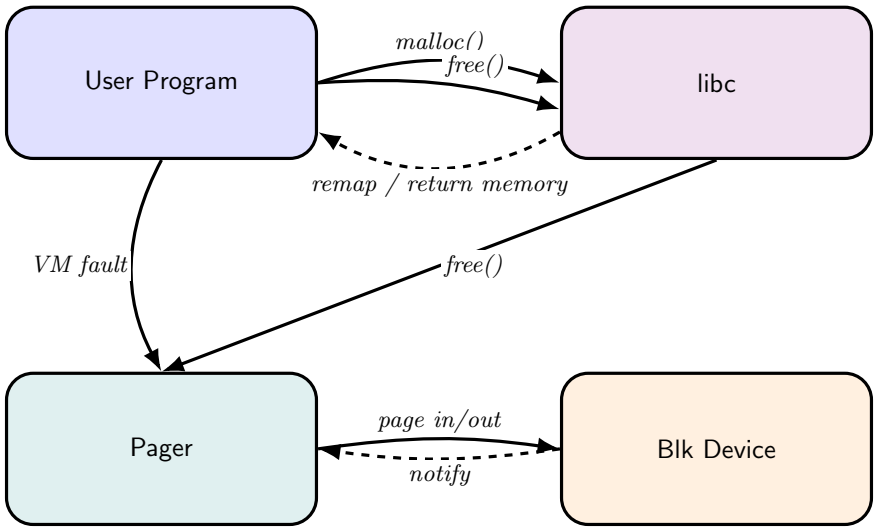


Figure 1: System Diagram for Pager and Memory Manager. Free is not yet implemented.

1.1 Design Problems

LionsOS is a static operating system. What this means is that all applications and memory regions etc. are loaded before runtime and these applications run for the lifetime of the system. This forms a problem regarding the implementation of the pager and memory manager on LionsOS as the memory manager is supposed to grant extra memory to a protection domain on request. However, where should this memory come from? It is important to note that the microkit currently does not support the retyping of memory in userland. This means that it is impossible without extensively changing microkit to even change the page tables during runtime.

We can instead premake page tables during compilation for a large area and map and unmap frames and do demand paging to make it seem that there is more memory than there actually is. The frames come from an *unbacked* memory region which is mapped into the pager's vspace and the pager through blk operations can implement demand paging. On compilation, the frame caps can be given to the pager for it to map to the clients' vspaces. *Note: this implementation is only a proof of concept design to run benchmarks to compare paging performance against Linux. A proper design would have the pager deal with untyped memory.*

1.2 Limitations of the Design

- There is a static upper limit of the number of protection domains that are supported by the pager, defined as `MAX_PDS`.
- Protection domains that are to be supported by the pager must be specified as 'children' of the pager in the system description file.
- The 'children' of the pager must have an ID less than `MAX_PDS`.
- The memory manager only allocates 4k size blocks.
- The Pager must be named 'pager' in the system description file.
- The number of available frames is fixed upon loading the system.
- The 'heap' allocated to by the memory manager always starts at 0x8000000000. (LionsOS libc reflects this).
- The 'heap' must be specified in the system description file as an unbacked memory region.
- Protection domains are singlethreaded.
- 'Child' protection domains of the pager do not have other faults except page faults.
- There is a static limit to the address space size - demand paging does not mean an infinite address space size.

1.3 Sdfgen

In order to support such a heap described in section 2.1, we need to implement a memory region which actually remains unmapped from the vspace of the protection domain of the client. In order to do this, we need to have a flag in the memory region object of the system description file to mark a memory region to remain unmapped. This memory region also needs to have its frame caps to be accessible by and mapped into the pager protection domain to do paging operations. In order to support this, I have added a new optional boolean field identified as 'backed'. By default memory regions are backed=True, and if they are backed=False, mappings are not done, only the page table structures are created, the memory region mapped to a protection domain named 'pager' and the frame caps passed to the protection domain named 'pager'. Protection domains can be marked as 'backed=False' which means that the stack of the protection domain is also unbacked and reliant on the pager using the same memory region's frames.

This is implemented in sdfgen in branch 'joshua/unbacked_pd' as an additional optional field in the constructor of `MemoryRegion`. It is also possible to bypass the sdfgen entirely by writing up the system description file by hand and including an attribute 'backed=false' in the memory region xml object.

1.4 Microkit

Microkit reads the SDF to load and run the system described in the SDF. I have modified microkit in order to implement the functionality described in the SDF.

1.4.1 Special Protection Domains

There are special protection domains which are named as below:

- pager

These are the required components to make the userland pager functional. The microkit when looping through the protection domains whilst building recognises these protection domains by their name string and implements special behaviour for these protection domains.

pager The pager needs to have all of the frames of the 'heaps' (and stacks if necessary) mapped into its vspace and needs to have all of the frame caps in order to map the frames to the client protection domains' vspaces. Microkit usually loops through the memory regions, creates frame caps for them and maps the frames using these frame caps into their respective protection domains as specified by the mappings in the sdf. The modification which I have made in microkit intercepts the mapping of frames and checks whether or not the memory region is 'backed', if not backed, the frame capability is stored in an array alongside with the id of the protection domain which the 'heap' is mapped to. The frame is also mapped into the pager. The array is written as a symbol into the pager's elf so that the pager has access to the frame caps.

1.4.2 Microkit Interface

Wrapper functions were implemented for seL4_ARM_map/unmap functionality in microkit.h to allow the pager to do mapping operations.

1.5 LionsOS

LionsOS is used instead of microkit in order to have easier access to drivers through SDDF.

1.5.1 User Application

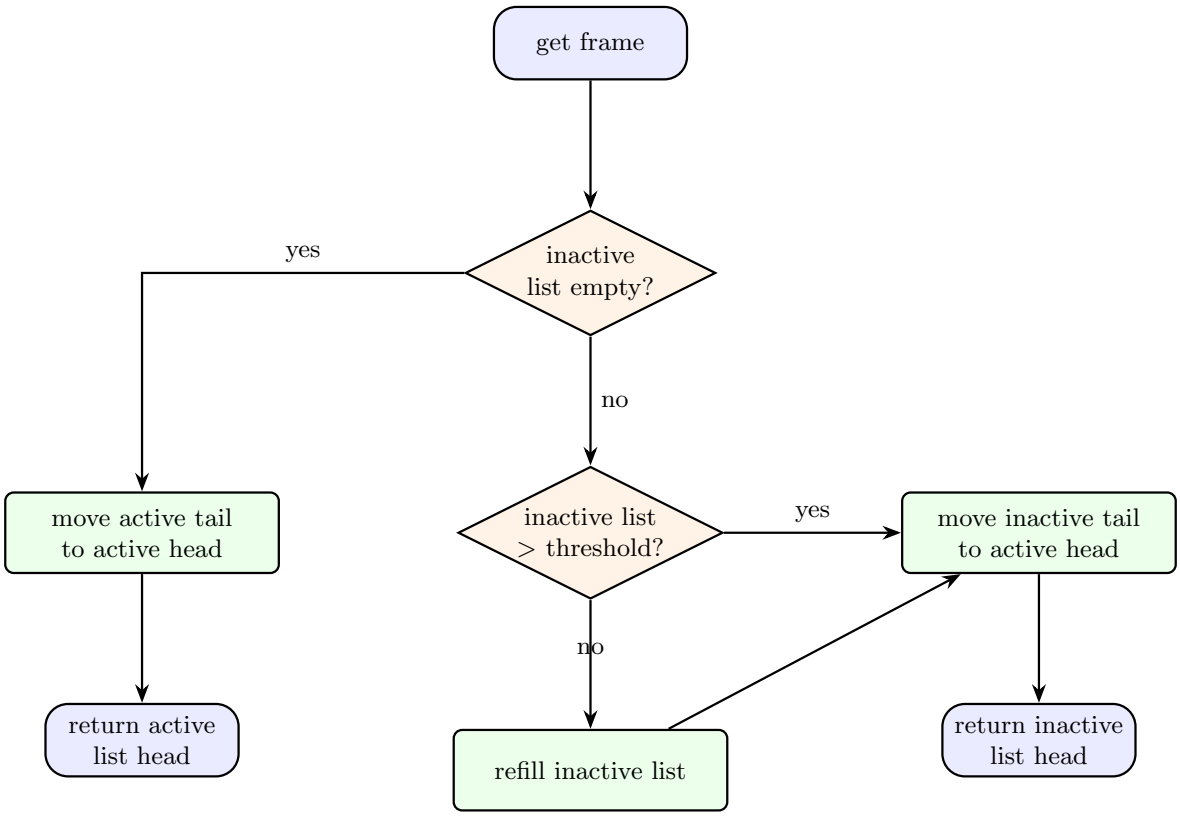
The user application should be configured with libc. I have modified the memory management code such that the morecore region starts at 0x8000000000 (just an arbitrarily hardcoded number) and thus can be supported by the pager. Normal malloc can be called and this will be supported by the pager. Free is not implemented for libc and also not supported by the pager. Frees will therefore leak memory.

1.5.2 Pager

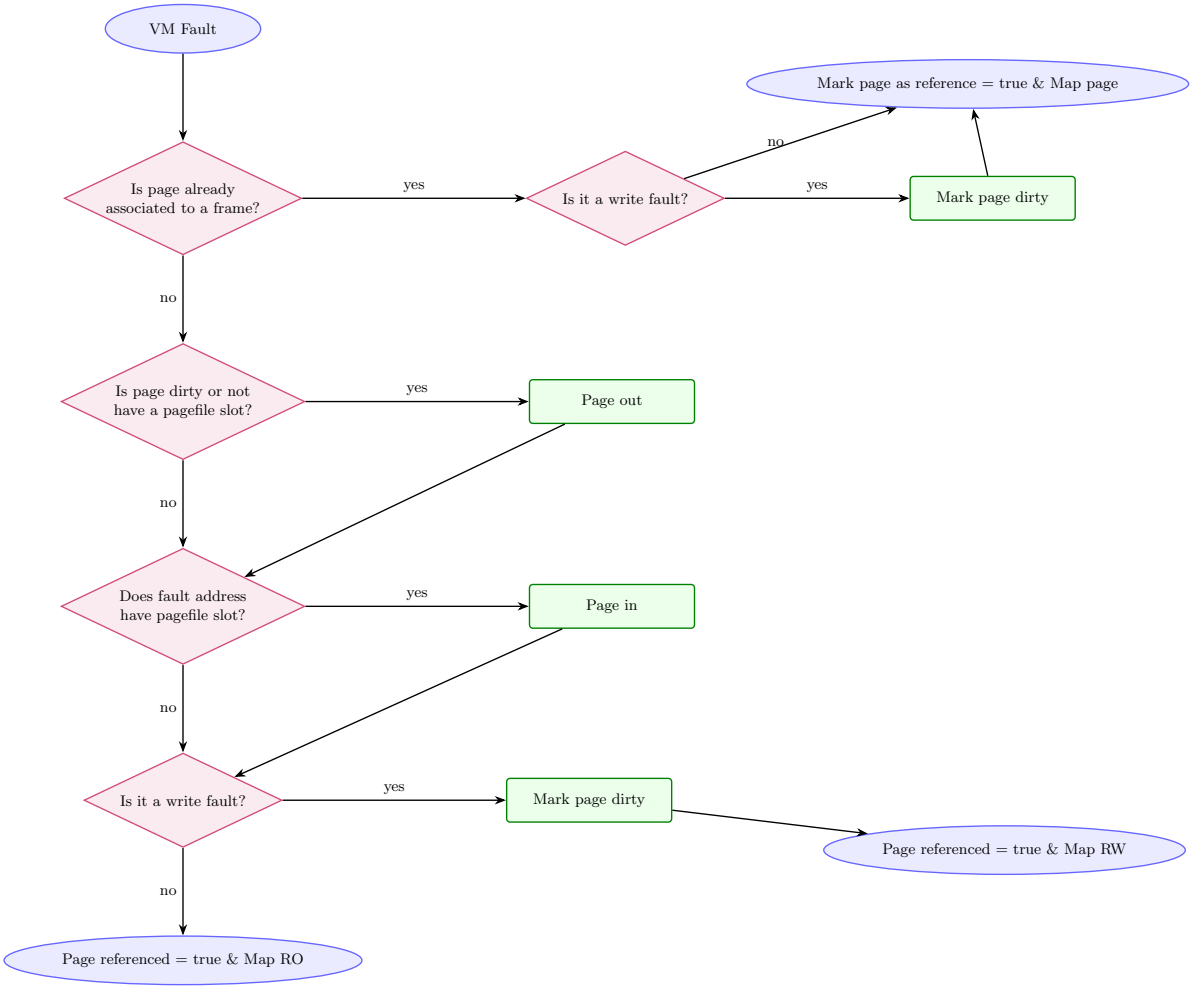
Data Structures There are two things the need to be kept track of: *pagefile slots & frames/pages.*

The way that I keep track of pagefile slots is through an array. The slot index represents the position of the slot in the file. Essentially I abstract the pagefile as an array. There is a counter which represents the highest slot number which has ever been used so if a new file slot is required, that number can be given and incremented up. There is also a stack which track the freed slots from when a page is freed. If a slot is requested, the stack is checked first then the highest slot number is incremented.

Instead of keeping track of page table entries as one normally would in a traditional pager implementation, since seL4 does not provide transparency into page tables, I have chosen to keep track of frame caps and a pseudo page table entry. pseudo page table entries are kept in an array of arrays, which are indexed by pd_idx and address / 4096. The struct carrying the frame caps are what is used in the page replacement algorithm. They are stored in three lists, a free list, an inactive list and an active list.



Two List Page Replacement The entry point for the two list page replacement algorithm is the `get_frame()` function which returns the next frame which could be used. This frame may or may not be already mapped to a page, but the pager takes care of that. The refill inactive function basically loops around the active list and if the current node's reference bit is set, sets this to false and unmaps and pushes to front of active list. If the current node is dereferenced then it is pushed to the front of the inactive list.



Paging Flow Paging follows the flow shown above, except that paging in and out operations are done asynchronously through the blk driver. We facilitate this by having a `page_continuations` array which allows the pager to index into using the child id and store the paging data which contains the frame, child id, fault address and a enum which tells the pager when notified that the blk operation is finished whether it was a page in or out. There is also a similar array of integers named `current_faults` also indexed by child id that stores the fault address to make sure that the current fault is not serviced multiple times. Since we assume that PD's are singlethreaded, there should only be one VM fault happening at any time for a single PD.